

CUDA 9

“If two wrongs don’t make a right, try three.”

Laurence J. Peter

We study a norm-setting, widely-used, and very successful programming model for GPUs. GPUs provide a lot of parallelism, and this is used to support a variety of different types of computation—not least related to Machine Learning and Large Language Models.¹

1: Recall Table 4.1 from Chapter 4.

Notes

- ▶ So far in this course, we focused on techniques for parallel and distributed computing over CPUs.
- ▶ We now turn to GPUs—which stands for “Graphics Processing Units”, but these days they’re used for tasks that go far beyond rendering graphics. GPUs evolved from providing a fixed-function graphics-processing hardware pipeline to providing a resource for highly parallel, general-purpose computing [25].
- ▶ In this course, GPUs are interesting to us because they provide us with a lot of hardware parallelism—we’ll analyze this in more detail soon. The parallelism in GPUs was (and is still) used to render a value for every single pixel on a display (in parallel).
- ▶ We focus on CUDA² since it is the best-known approach for programming GPUs.

[25]: Fatahalian et al. (2008), *A closer look at GPUs*

2: In case you’re curious, CUDA stands for “Computer Unified Device Architecture”.

Alternatives to CUDA

For other approaches, in no particular order, see OpenACC, OpenCL, ISPC, HIP, ROCm, OneAPI, SYCL, Kokkos, JAX, and also OpenMP (which we saw in Chapter 5 but didn’t cover its GPU targeting).

- ▶ CUDA is primarily associated with GPUs from a specific vendor—the one you’re thinking of—who originally developed this approach, but these days there are adaptation tools that can get CUDA code to run on GPUs from different vendors.
- ▶ For more on this topic see Chapter 42 of Volume 2 of *Parallel Programming for Science and Engineering*.³
- ▶ How do we characterize a GPU? We can use the criteria from other computing resources such as CPUs. For an example, let’s take the RTX-6000 GPU that’s based on NVIDIA’s Ada design. How is that specific GPU characterized?
 - An important dimension is *compute*. According to NVIDIA’s public website,⁴ the RTX-6000 can reach 91.1 teraflops.⁵ More recent GPUs can reach *much* higher rates—hundreds of teraflops.

3: https://github.com/VictorEijkhout/TheArtofHPC_pdfs/blob/d5cb3bffa41d54125a729fdb97e307c80de44da1/vol2/EijkhoutParallelProgramming.pdf

4: <https://www.nvidia.com/en-us/products/workstations/rtx-6000/>

5: That’s 10^{12} FLOPS: *floating-point operations per second*

6: Based on the information from the previous point, note that transfers to/from host memory are 30× slower than transfers to/from on-board memory.

7: Back in Chapter 1, we saw the CM-1 (Fig 1.5b) capable of up to 10 gigaflops, CM-5 (Fig 1.6) capable of up to 131 gigaflops, and Stampede2 (Fig 1.7) capable of up to 18-petaflops.

[26]: M. Kiernan et al. (1995), *Implementation of IEEE Floating-point Arithmetic on the Cray T90 System*

8: Standardized as IEEE-754. <https://standards.ieee.org/ieee/754/6210/>

[27]: Patel et al. (2022), *In-network fractional calculations using P4 for scientific computing workloads*

9: For comparison, the FASTER cluster at TA&MU has 11,520 CPU cores distributed across 180 nodes (but it also has a bunch of GPUs): <https://hprc.tamu.edu/kb/User-Guides/FASTER/Hardware/>

10: It appears that the data sheet also refers to those cores as “Shading Units”.

11: The 4×32-way SIMT is a standard pattern that this vendor settled upon for SMs.

12: That 32-way SIMT is also called a “warp” since it’s an orthogonal grouping of parallel resources—as in how a warp is a group of parallel threads. But don’t get too caught up in the metaphor: a different vendor refers to this idea as a “wavefront”. However we might call it, think of a warp as a unit in which threads are scheduled together.

Bottlenecks

Often, workloads are *not* compute-bound—they’re often memory-bound (i.e., insufficient memory) or IO-bound (i.e., relatively slow communication with other devices).

- Another important dimension is on-board *memory* and the *bandwidth* between the GPU and that memory. This model has 48GB of memory and 960 GB/s bandwidth.
 - Another important dimension is *IO* (Input/Output). This model has a PCIe 4.0 x16 interface, which has a bandwidth of 31.51 GB/s.⁶
 - Another dimension is *power*. This GPU has a maximum utilization of 300W. Through overclocking, GPU cards can reach 600W.
- ▶ We saw earlier that performance is rated in FLOPS: *floating-point operations per second*.⁷
 - ▶ But what does this mean? There are several floating-point encodings! We typically see FP32 and FP64—which we call *single-* and *double-*precision respectively. But we also encounter others: including TF32 (TensorFloat 32), FP16, FP8, BF16, the 80-bit extended precision on the 8087 co-processor, and 128 bit precision on Cray T90 [26]. FLOPS refers to *single-precision* floating point.⁸
 - ▶ It’ll help us to brush up on the IEEE-754 floating-point encoding techniques for reals. In case you’re curious to find out more, there are also so-called fixed-point encodings [27].
 - ▶ In addition to the programming model for GPUs, it’ll help us to learn a bit about GPU architecture. A GPU is organized into a set of tiles that provide different functions. Most tiles group together cores and memory.
 - ▶ Download the data sheet from the product page linked earlier,⁴ and you’ll see that the sheet mentions that the GPU includes “18,176 CUDA® cores”. That’s a good deal more cores than in server-class CPUs! But are we talking about the same type of core?⁹

Core count

How many cores are there in a typical server-class CPU today?
How does that compare with the number of cores in your phone?
And your laptop?

Besides those cores, the data sheet also mentions “142 third-generation RT Cores, 568 fourth-generation Tensor Cores”—these are intended for more specialised processing: ray-tracing and machine-learning.

- ▶ Let’s unpack how those 18,176 cores are counted.¹⁰ GPUs in the series of the RTX 6000 are structured into a hierarchy of resources. The closest thing to a CPU core in this hierarchy is a Streaming Multiprocessor (SM). From the data sheet we that “SM Count” is 142. We don’t see this in the data sheet,¹¹ but each SM is said to have 4 sub-cores and 32-way SIMT,¹² where SIMT is “Single-Instruction-Multiple-Thread”.
- ▶ A “thread” in this context is different than what we encountered previously. In this context, a thread is synonymous with a “streaming processor” or SP. You can think of a thread as a lane in SIMD. Each

thread in a warp proceeds in lock-step—just as processing on vector registers would.

- ▶ Putting all that together, we have 142 SMs, each providing 4 sub-cores and 32 SPs.¹³ $142 \times 4 \times 32 = 18,176$ —which is the number we sought to unpack.
- ▶ GPUs have other hierarchical structural groupings, but they're not important for our purposes in this course. For example, you might encounter expressions such as Graphics Processing Cluster (GPC) and Texture Processing Cluster (TPC).
- ▶ There are three types of memory that we can use: thread-local memory (fastest), block-local memory (shared between threads), and device-level memory (slowest). The data sheet tells us that there is 128 KB (per SM) of L1 Cache, and 96 MB of L2 Cache.
- ▶ Let's imagine that we have a program that has been compiled for a GPU. How do we start that program? We write an accompanying program that runs on the CPU and sets up data transfers to the GPU's memory and queues work for the GPU. You typically have thread running on the CPU that is managing the workload for the GPU. This is an example of a system that relies both on distributed and parallel computing.¹⁴
- ▶ Finally, we turn to the **programming model**! Two connections show up with other parts of the course:
 1. A GPU relies on a paradigm that's similar to vector/array processing, which we encountered in SIMD (Chapter 7).
 2. Every CUDA thread is assigned a unique identifier composed of its block number and thread number. This identifier is used for SPMD-based processing, which we encountered in MPI (Chapter 8).
- ▶ We looked at the CUDA example that comes with the FABRIC notebook. We caught a glimpse of the following functions:

```
cudaMalloc( (void**)&ad, csize );
cudaMalloc( (void**)&bd, isize );

cudaMemcpy( ad, a, csize, cudaMemcpyHostToDevice );
cudaMemcpy( bd, b, isize, cudaMemcpyHostToDevice );
...
cudaMemcpy( a, ad, csize, cudaMemcpyDeviceToHost );

cudaFree( ad );
cudaFree( bd );
```

- ▶ Note the distinction between **host** and **device** (and that there can be multiple devices—i.e., GPUs). Code is annotated based on where it's intended to be called and run: `__host__` (which is the default when a C function is not annotated), `__device__`, and `__global__` (can be called from either the host or the device, and runs on the device).¹⁵
- ▶ We saw how to start a **kernel**—which is like the “P” in SPMD—and CUDA's `kernel<<<grid, block>>>(parameters)` notation. Think of a kernel as a function that you call from the CPU, and that executes on the GPU. The `<<<grid, block>>>` is called the *execution configuration* of the kernel.
- ▶ The *grid* parameter describes the number of blocks, and the *block* parameter describes the number of threads. The grid and block parameter values can be produced using the constructor of `dim3`—we saw this in the first CUDA example we looked at.

13: Look at the whitepaper for the Ada microarchitecture (particularly page 11) to get more details: https://images.nvidia.com/aem-dam/en-zz/Solutions/technologies/NVIDIA-ADA-GPU-PROVIZ-Architecture-Whitepaper_1.1.pdf

14: To have the code running on the host wait until the code running on the device has completed, we call `cudaDeviceSynchronize()`; in the code that's running on the CPU.

15: A C function can be annotated with `__host__` and `__device__`—what would that mean?

- ▶ A kernel's execution is formed of a grid of blocks of threads. Threads in a block can synchronize, but blocks in a grid cannot.
- ▶ Recall SMs from earlier. A block (set of threads) is assigned to a single SM. An SM can be assigned several blocks, which are executed in parallel.
- ▶ Similarities with MPI:
 - Grids and blocks can be arranged in different ways relative to each other—this is similar to how we can structure *communicators* in MPI.
 - The size of the kernel can be varied by changing the `<<<L,M>>>` parameter, while in MPI we can vary the host configuration file and/or use the `-n X` parameter.
 - Threads can find out their identity at runtime and behave accordingly—this is similar to how we use *rank* in MPI.
- ▶ At runtime, threads can query the following: `gridDim.x/y/z`, `blockDim.x/y/z`, `blockIdx.x/y/z`, `threadIdx.x/y/z`, and `warpSize`
- ▶ Computing the a thread's unique ID involves calculations over (some of) the runtime values from the previous point. For instance: `blockIdx.x * blockDim.x + threadIdx.x`.
- ▶ Putting together some of what we've seen so far, these are examples "Minimal Kernels" that NVIDIA had provided when initially describing CUDA:

```

1 __global__ void kernel( int *a )
2 {
3     int idx = blockIdx.x*blockDim.x + threadIdx.x;
4     a[idx] = 7;
5 }
6
7 __global__ void kernel( int *a )
8 {
9     int idx = blockIdx.x*blockDim.x + threadIdx.x;
10    a[idx] = blockIdx.x;
11 }
12
13 __global__ void kernel( int *a )
14 {
15     int idx = blockIdx.x*blockDim.x + threadIdx.x;
16     a[idx] = threadIdx.x;
17 }

```

16: The second example involves a kernel that prints some output. Isn't it weird that you can call `printf()` from the GPU? What do you think is going on?

- ▶ For our second example of a CUDA program,¹⁶ and to obtain a description of the **toolchain**, we looked at <https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/nvcc.html>
- ▶ Our third example generalizes the second:

```

1 #include <stdio.h>
2
3 __global__ void hello_cuda() {
4     //printf("hello from (%d, %d, %d)\n", threadIdx.x,
5         threadIdx.y, threadIdx.z);
6     printf("hello from (%d / %d)\n", threadIdx.x, blockIdx.x)
7     ;
8 }
9
10 int main()
11 {

```

```

10 | hello_cuda<<<2,5>>>();
11 | cudaDeviceSynchronize();
12 | return 0;
13 | }

```

```

ubuntu@gpu-node:~/test$ /usr/local/cuda/bin/nvcc hello2.cu
↳ -L /usr/local/cuda/lib -lcudart -o hello2
ubuntu@gpu-node:~/test$ ./hello2
hello from (0 / 1)
hello from (1 / 1)
hello from (2 / 1)
hello from (3 / 1)
hello from (4 / 1)
hello from (0 / 0)
hello from (1 / 0)
hello from (2 / 0)
hello from (3 / 0)
hello from (4 / 0)

```

- In addition to the `cudaMalloc/cudaMemcpy/cudaFree` approach we saw earlier, we can use the *unified memory* abstraction. That is, we can allocate memory that can be accessed both from the host and the device. We'll use this in our next example. This avoids the `cudaMemcpy` step—but not `cudaFree`.
- In the next example, the kernel invocation starts out as: `compute<<<1, 1>>>(arr_size, arr);`. Let $N > 1$, and use N as a proxy measure of parallelism.¹⁷ Then think about what happens in the following configurations:
 1. `compute<<<N, 1>>>(arr_size, arr);`
This approach is a bit wasteful—only a single thread is run per block.
 2. `compute<<<1, N>>>(arr_size, arr);`
On the other hand, this approach can be constrained by the memory needed by threads in single block that they're assigned to (and which is mapped to a single SM).
 3. `compute<<<M, N>>>(arr_size, arr);` where $M > 1$
- For our fourth example, we examine this code and note the possibility of data races:

```

1 | #include <stdio.h>
2 |
3 | __host__
4 | void print_contents(int arr_size, int *arr, const char *
   | caption)
5 | {
6 |     int stride = 80;
7 |     printf("%s", caption);
8 |     for (int i = 0; i < arr_size; i += stride) {
9 |         for (int j = i; j < i + stride; j++) {
10 |             if (j >= arr_size) break;
11 |             printf("%d, ", arr[j]);
12 |         }
13 |         printf("\n");
14 |     }
15 | }
16 |
17 | __global__
18 | void

```

¹⁷ What is the maximum value of N ? It depends on whether it represents the size of grids or blocks, and for either case it depends on the GPU that you have. See this table for example limits for NVIDIA devices across generations: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#features-and-technical-specifications> For example, you can have at most 1024 threads per block.

18: For an example of implementing matrix multiplication with tiling (to optimize the computation) see: <https://github.com/AdepojuJeremy/CUDA-120-DAYS--CHALLENGE/blob/main/daily-updates/day-4-Thread-Hierarchy%3A-Grids-%26-Blocks.md#b-matrix-multiplication-example>

► Ideas for exercises:

- Matrix addition.
- Reversing an array.
- Matrix multiplication.¹⁸

► This is an example program that gets information about a GPU device:

```
1 // /usr/local/cuda/bin/nvcc test-world.cu -L /usr/local/
  cuda/lib -lcudart -o test-world
2
3 #include <stdio.h>
4
5 #define DEVNO 0
6
7 int
8 main()
9 {
10     cudaSetDevice(DEVNO);
11
12     int cap_maj = -1;
13     int cap_min = -1;
14     cudaDeviceGetAttribute(&cap_maj,
15         cudaDevAttrComputeCapabilityMajor, DEVNO);
16     cudaDeviceGetAttribute(&cap_min,
17         cudaDevAttrComputeCapabilityMinor, DEVNO);
18     printf("cap_maj=%d, cap_min=%d\n", cap_maj, cap_min);
19
20     int pci_bus = -1;
21     int pci_device = -1;
22     cudaDeviceGetAttribute(&pci_bus, cudaDevAttrPciBusId,
23         DEVNO);
24     cudaDeviceGetAttribute(&pci_device,
25         cudaDevAttrPciDeviceId, DEVNO);
26     printf("pci_bus=%d, pci_device=%d\n", pci_bus, pci_device
27         );
28
29     int mp_count = -1;
30     int warp_size = -1;
31     int max_threads_per_block = -1;
32     int max_threads_per_mp = -1;
33     cudaDeviceGetAttribute(&mp_count,
34         cudaDevAttrMultiProcessorCount, DEVNO);
35     cudaDeviceGetAttribute(&warp_size, cudaDevAttrWarpSize,
36         DEVNO);
37     cudaDeviceGetAttribute(&max_threads_per_block,
38         cudaDevAttrMaxThreadsPerBlock, DEVNO);
39     cudaDeviceGetAttribute(&max_threads_per_mp,
40         cudaDevAttrMaxThreadsPerMultiProcessor, DEVNO);
41     printf("mp_count=%d, warp_size=%d, max_threads_per_block
42         =%d, max_threads_per_mp=%d\n", mp_count, warp_size,
43         max_threads_per_block, max_threads_per_mp);
44
45     int memory_clock_rate = -1; // in kilohertz
46     int global_memory_bus_width = -1; // in bits
47     int l2_cache_size = -1; // in bytes
48     int unified_addressing = -1; // 1=shared, 0=otherwise
49     int ecc_enabled = -1; // 1=enabled, 0=disabled or
50         unsupported
```



```

39  int max_shared_memory_per_block = -1; // in bytes
40  int total_constant_memory = -1; // in bytes
41  int integrated = -1; // 1=integrated, 0=otherwise
42  int can_map_host_memory = -1; // 1=can map, 0=otherwise
43  cudaDeviceGetAttribute(&memory_clock_rate,
44      cudaDevAttrMemoryClockRate, DEVNO);
45  cudaDeviceGetAttribute(&global_memory_bus_width,
46      cudaDevAttrGlobalMemoryBusWidth, DEVNO);
47  cudaDeviceGetAttribute(&l2_cache_size,
48      cudaDevAttrL2CacheSize, DEVNO);
49  cudaDeviceGetAttribute(&unified_addressing,
50      cudaDevAttrUnifiedAddressing, DEVNO);
51  cudaDeviceGetAttribute(&ecc_enabled,
52      cudaDevAttrEccEnabled, DEVNO);
53  cudaDeviceGetAttribute(&max_shared_memory_per_block,
54      cudaDevAttrMaxSharedMemoryPerBlock, DEVNO);
55  cudaDeviceGetAttribute(&total_constant_memory,
56      cudaDevAttrTotalConstantMemory, DEVNO);
57  cudaDeviceGetAttribute(&integrated, cudaDevAttrIntegrated
58      , DEVNO);
59  cudaDeviceGetAttribute(&can_map_host_memory,
60      cudaDevAttrCanMapHostMemory, DEVNO);
61  printf("memory_clock_rate=%d, global_memory_bus_width=%d,
62      l2_cache_size=%d, unified_addressing=%d, ecc_enabled=%d,
63      max_shared_memory_per_block=%d,
64      total_constant_memory=%d, integrated=%d,
65      can_map_host_memory=%d\n", memory_clock_rate,
66      global_memory_
67      bus_width, l2_cache_size, unified_addressing, ecc_enabled,
68      max_shared_memory_per_block, total_constant_memory,
69      integrated, can_map_host_memory);
70
71  return 0;
72 }

```

```

ubuntu@gpu-node:~$ ./test-world
cap_maj=7, cap_min=5
pci_bus=7, pci_device=0
mp_count=72, warp_size=32, max_threads_per_block=1024,
    ↳ max_threads_per_mp=1024
memory_clock_rate=6501000, global_memory_bus_width=384,
    ↳ l2_cache_size=6291456, unified_addressing=1,
    ↳ ecc_enabled=1, max_shared_memory_per_block=49152,
    ↳ total_constant_memory=65536, integrated=0,
    ↳ can_map_host_memory=1

```

- ▶ There's a *lot* more to CUDA. Immediate next topics include: variable annotations like `__shared__` to influence placement, shared memory and synchronization across threads,¹⁹ and optimizing the number of threads and blocks based on our knowledge of the SMs and warps on the device.
- ▶ Go back to the LLM example we had glimpsed during the analysis described on page 30, and work through the code. (Look at `ya lm/src/infer.cu`.)

19: `__syncthreads()` in CUDA creates a barrier between threads in a block—somewhat similar to `#pragma omp barrier` in OpenMP (from Chapter 5).

Food for thought

Read this thought-provoking piece by Chamberlain:
<https://chapel-lang.org/blog/posts/30years/>

Then think about how CUDA features in that article. How do you see the field evolving?

2026/04/21 11:00:00
Draft of